

Redes Neurais e Aprendizagem Profunda

Keras



Instalar o software

1. Instalar o tf.keras. Pode usar o comando

```
python3 -m pip install -U tensorflow
```

2. Pode verificar a instalação numa shell Python com os seguintes comandos

```
>>> from tensorflow import keras
>>> keras.__version__
'2.11.0'
```

Construir um classificador de imagens

O Fashion MNIST é um conjunto com 70000 imagens de 28*28 pixels e 10 classes, cujas imagens representam itens de moda. Pode ver exemplos na figura seguinte.



1. O Keras possui funções para abrir e carregar conjuntos de dados comuns, sendo o Fashion MNIST um deles. Pode fazê-lo através do código seguinte:

```
fashion_mnist = keras.datasets.fashion_mnist
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
```

Ao contrário do Scikit-Learn, no Keras as imagens são representadas por um array 28*28 (2 dimensões) e a intensidade dos pixels é representada por um inteiro (entre 0 e 255). Verifique a forma e o tipo do conjunto de treino usando as variáveis `shape` e `dtype`.

2. A partir do conj de treino, crie um conjunto de validação com 5000 exemplos e escale a intensidade dos pixels para um real no intervalo [0,1]

3. Utilize a atribuição seguinte para dar nomear cada uma das classes

```
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal",
               "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

4. A instrução seguinte cria uma rede neuronal multi-camada. `Sequential()` é o modelo mais simples do

Keras, composto por uma pilha de camadas ligadas sequencialmente. A camada `Flatten` converte cada imagem num vetor, sendo necessário indicar o formato de entrada (28*28).

```
model = keras.models.Sequential()
model.add(keras.layers.Flatten(input_shape=[28, 28]))
model.add(keras.layers.Dense(300, activation="relu"))
model.add(keras.layers.Dense(100, activation="relu"))
model.add(keras.layers.Dense(10, activation="softmax"))
```

Quantas são as camadas escondidas? Quantos neurónios existem em cada camada e qual a correspondente função de ativação? Quantos são os neurónios da camada de saída? Porquê?

5. A função `summary()` mostra informação sobre cada uma das camadas da rede. Quantos parâmetros (pesos) tem a primeira camada escondida? Porquê?

6. A lista das camadas da rede é obtida através da variável `layers`. Os métodos `get_weights()` e `set_weights()` permitem obter e alterar os pesos de uma camada. Para uma camada `Dense` inclui os pesos das ligações e o vetor bias. usar, por exemplo,

```
weights, biases = hidden1.get_weights()
```

7. O método `compile()` permite especificar a função de perda e o otimizador. É possível adicionar as outras métricas a calcular durante o treino e avaliação.

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd", metrics=["accuracy"])
```

Confirme na documentação do Keras o que indica e o otimizador `sgd` e a função de perda `sparse_categorical_crossentropy`.

8. Para treinar o modelo utiliza-se o método `fit()` indicando o nº de épocas e o conjunto de validação:

```
history = model.fit(X_train, y_train, epochs=30,
                   validation_data=(X_valid, y_valid))
```

O método `fit()` devolve um objeto `History` com os parâmetros de treino (`history.params`), a lista de épocas (`history.epoch`) e um dicionário (`history.history`) com a perda e outras métricas medidas no final de cada época sobre os conjuntos de treino e teste.

Verifique a informação mostrada ao longo do treino.

9. O dicionário pode ser usado para mostrar as curvas de aprendizagem, usando um `DataFrame` `pandas` e o método `plot()`.

```
import pandas as pd
import matplotlib.pyplot as plt
pd.DataFrame(history.history).plot(figsize=(8, 5))
plt.grid(True)
plt.gca().set_ylim(0, 1)
plt.show()
```

Confirme na documentação o que faz cada um dos métodos.

10. Altere os hiperparâmetros (`learning_rate`, otimizador, nº de camadas, nº de unidades por camada, tipos de função de ativação, `batch_size`) para tentar obter um modelo melhor.

11. Estime o erro de generalização através do método `evaluate()`.
12. O método `predict()` permite fazer previsões sobre novos exemplos. Teste com um exemplo do conjunto de treino? O que é que o modelo prevê?

TensorBoard

O TensorBoard é uma ferramenta de visualização interativa que permite visualizar as curvas de aprendizagem durante o treino, compará-las entre várias execuções, visualizar o grafo de computação, analisar as estatísticas de treino, visualizar imagens geradas pelo modelo, visualizar dados multidimensionais projetados em 3D e agrupados automaticamente, para além de outras funcionalidades. Esta ferramenta é instalada automaticamente com o TensorFlow.

Para usá-lo, é preciso modificar o programa para que ele gere os dados para visualização em ficheiros de log binários especiais (event files). O servidor do TensorBoard monitora o log, e atualiza as visualizações com os novos dados. Normalmente, aponta-se o servidor TensorBoard para uma diretoria de log raiz e configura-se o programa para que grave em subdiretórias diferentes cada vez que é executado. Desta forma, a mesma instância do servidor TensorBoard permite visualizar e comparar dados de várias execuções.

As instruções seguintes criam um diretoria de logs para uso do TensorBoard e uma pequena função que cria uma subdiretoria base na data e hora atuais (pode adicionar outra informação no nome da diretoria para facilitar a visualização no TensorBoard):

```
import os
root_logdir = os.path.join(os.getcwd(), "my_logs")
def get_run_logdir():
    import time
    run_id = time.strftime("run_%Y_%m_%d-%H_%M_%S")
    return os.path.join(root_logdir, run_id)
run_logdir = get_run_logdir()
```

O código seguinte cria um TensorBoard callback que cria a diretoria de logs e os ficheiros de eventos durante o treino onde escreve os sumários:

```
tensorboard_cb = keras.callbacks.TensorBoard(run_logdir)
history = model.fit(X_train, y_train, epochs=30,
                    validation_data=(X_valid, y_valid),
                    callbacks=[tensorboard_cb])
```

Depois é só chamar o TensorBoard a partir da diretoria onde está o projeto:

```
$ tensorboard --logdir=./my_logs --port=6006
```

abrir o browser e "ir" para <http://localhost:6006>.

Alternativamente é possível usar o TensorBoard a partir do Jupyter através dos seguintes comandos:

```
%load_ext tensorboard
%tensorboard --logdir=./my_logs --port=6006
```

Explore o TensorBoard usando as experiências realizadas no ponto 10.